TU BIBLIOTECA · ORDENADA PARA BACKEND JAVA → ARQUITECTURA FINTECH

Ruta de lectura

Los 21 libros de tu carpeta en el orden en que conviene leerlos, pensado para alguien que está aprendiendo Spring Boot desde cero, busca su primer trabajo y apunta a arquitecto en el sector financiero. La línea principal tiene 5 fases; IA y bajo nivel van en ramales paralelos para que no frenen la línea principal. Al final está lo que te falta para llegar a arquitecto senior: dominio financiero, liderazgo, comunicación y estrategia.

Spring Boot · Oficio · Primer trabajo · Arquitectura · Distribuidos · Ramal IA · Ramal bajo nivel

Un libro principal a la vez

Uno que leés de corrido y, como mucho, uno de consulta abierto al lado. Las fases 2 y 3 sí van en paralelo entre sí.

Cada capítulo, a tu código

Cada estación trae un ejercicio concreto sobre Billetera Clara o apifintech. Si no lo podés hacer, el capítulo todavía no quedó.

Notas a mano, con un caso

En Obsidian, por capítulo: la idea en una frase y un ejemplo concreto tuyo. Sin ejemplo propio, la nota no sirve para repasar.

1

Spring Boot de verdad

Ahora, en paralelo con la diplomatura. Objetivo: entender qué hace Spring por debajo de cada anotación que ya usás.



ESTACIÓN 1

☐ Leído

Spring Start Here

Laurențiu Spilcă · Manning · ~400 págs

Está escrito para quien arranca Spring desde cero. Explica el contexto de Spring, los beans y la inyección de dependencias antes de llegar a REST, acceso a datos, transacciones y tests. Es exactamente lo que Spring Boot te esconde.

> **Llevalo a tu código:** Por cada capítulo, buscá en apifintech dónde aparece lo que leíste (@Service, @Transactional, @RestController) y anotá qué pasaría si sacaras esa anotación. Después probalo.

Edición: Tu PDF es la 1ª edición (Spring 5). La 2ª edición, con Spring 7 y Spring Boot 4, está en acceso anticipado en Manning con publicación estimada para diciembre de 2026. Los conceptos de la 1ª siguen valiendo.



ESTACIÓN 2

☐ Leído

Spring in Action

Craig Walls · Manning · ~520 págs

Segunda pasada, más amplia: Spring Data, Spring Security, mensajería, programación reactiva y Actuator. No hace falta leerlo de corrido; usalo por tema cuando lo necesites.

> **Llevalo a tu código:** Antes de agregar login a Billetera Clara, leé los capítulos de Spring Security y protegé un solo endpoint. Recién después extendelo al resto.

Edición: La 6ª edición (2022) es la más reciente. Es anterior a Spring Boot 3, así que vas a ver paquetes javax.* donde hoy se usa jakarta.*.

Oficio: código que otro puede leer

2 Oficio: código que otro pueda leer

Cuando ya armás un CRUD sin guía. Es la base para leer repos grandes como Kill Bill o Fineract.



ESTACIÓN 3

Leído

A Philosophy of Software Design

John Ousterhout · ~190 págs

El más corto y el que más cambia cómo pensás el diseño. Su idea central: el enemigo es la complejidad, y la combatís con módulos profundos, que esconden mucho detrás de una interfaz chica.

> **Llevalo a tu código:** Tomá el servicio de cuentas de apifintech: contá sus métodos públicos y cuánto hace cada uno. Si exponés cinco métodos para lo que el controlador ve como una sola operación, juntalos.

Edición: Conviene la 2ª edición (2021), que revisa su postura sobre comentarios y métodos cortos.



ESTACIÓN 4

Leído

Refactoring

Martin Fowler · ~450 págs

Un catálogo de cambios chicos y seguros para mejorar código sin romperlo, siempre con tests verdes entre paso y paso. De paso, te obliga a escribir tests, uno de tus puntos flojos.

> **Llevalo a tu código:** Aplicá tres refactorings en Billetera Clara: Extract Function, Introduce Parameter Object y Replace Conditional with Polymorphism. Corré los tests después de cada paso.

Edición: La 2ª edición (2018) tiene los ejemplos en JavaScript; la 1ª (1999) los tenía en Java. Las técnicas se pasan a Java sin problema.



ESTACIÓN 5

Leído

Working Effectively with Legacy Code

Michael Feathers · ~450 págs

Es justo la habilidad que querés desarrollar: modificar código que no escribiste sin romperlo. Te enseña a encontrar "seams" (puntos donde podés cortar dependencias) y a escribir tests de caracterización.

> **Llevalo a tu código:** En el próximo repo de tu ruta (Joda-Money, Kill Bill), antes de tocar una clase escribí un test que fije lo que hace hoy, aunque parezca raro. Recién entonces cambiala.

Edición: Ejemplos en Java, C++ y C. Es de 2004 pero no envejeció.



ESTACIÓN 6 · CONSULTA

Leído

Code Complete

Steve McConnell · 2ª ed. · ~900 págs

Una enciclopedia de la construcción de software. No lo leas de tapa a tapa: tenelo como consulta para nombres de variables, rutinas, programación defensiva y depuración.

> **Llevalo a tu código:** Usá sus checklists al final de cada capítulo como revisión antes de hacer un commit grande o pedir code review.

Edición: La 2ª edición (2004) es la vigente. Los ejemplos mezclan varios lenguajes, incluido Java.

3 Primer trabajo

En paralelo con la fase 2: un poco cada día. Junto con tus PDFs de preguntas y la prueba técnica.



ESTACIÓN 7

Leído

Cracking the Coding Interview

Gayle Laakmann McDowell · ~700 págs

Estructuras de datos, algoritmos y cómo encarar una entrevista técnica. Leé las explicaciones de cada tema y resolvé los problemas en Java.

> **Llevalo a tu código:** Un problema por día, cronometrado, en Java y sin IA. Anotá en Obsidian solo el patrón que lo resolvía (dos punteros, hash map, BFS).

Edición: En enero de 2025 salió Beyond Cracking the Coding Interview (McDowell, Mroczka, Lerner y Mamano), más actual. Si lo conseguís, usá ese como principal.



ESTACIÓN 8

Leído

Programming Pearls

Jon Bentley · 2ª ed. · ~250 págs

Columnas cortas sobre pensar el problema antes de programar: estimaciones rápidas, búsqueda binaria correcta, rendimiento. Es un buen puente hacia system design.

> **Llevalo a tu código:** Hacé la estimación al estilo del libro: si Billetera Clara tuviera 100.000 usuarios con 5 movimientos diarios, ¿cuántas escrituras por segundo en el pico? ¿Cuánto disco por año?

4 De programador a arquitecto

Cuando ya tenés un proyecto propio con varias capas. Acá aparece el dominio financiero de la mano de DDD.



ESTACIÓN 9

Leído

Head First Software Architecture



Gandhi, Richards y Ford · 2024 · ~500 págs

La entrada más amable a la arquitectura: características, decisiones y estilos (en capas, monolito modular, microkernel, microservicios, orientado a eventos), con muchos dibujos y ejercicios.

> **Llevalo a tu código:** Elegí qué estilo tiene hoy tu proyecto y escribí en una página por qué, con las características que más importan en una billetera (consistencia, seguridad, auditabilidad).



ESTACIÓN 10

☐ Leído

Fundamentals of Software Architecture

Mark Richards y Neal Ford · ~420 págs

El mismo terreno que Head First, más profundo y formal. Agrega cómo medir la modularidad, cómo documentar decisiones y qué hace un arquitecto en el día a día.

> **Llevalo a tu código:** Escribí tu primer ADR (registro de decisión de arquitectura) para una decisión real de Billetera Clara, por ejemplo base relacional o no.

Edición: Hay 2ª edición de marzo de 2025 con cinco capítulos nuevos, incluidos IA generativa, nube y topologías de datos y equipos.



ESTACIÓN 11

☐ Leído

Clean Architecture

Robert C. Martin · ~430 págs

La regla de dependencias, los puertos y adaptadores, y SOLID aplicado a componentes. Leelo con espíritu crítico: Ousterhout discute varias ideas del mismo autor, y esa tensión te enseña a decidir.

> **Llevalo a tu código:** Dibujá las dependencias entre paquetes de apifintech. ¿Tu dominio importa algo de Spring o de JPA? Si sí, probá separar una entidad de dominio de su entidad JPA.



ESTACIÓN 12 · CONSULTA

☐ Leído

Patterns of Enterprise Application Architecture

Martin Fowler · ~530 págs

Los patrones que están dentro de JPA y Hibernate, con nombre propio: Unit of Work, Identity Map, Data Mapper, Lazy Load, Optimistic Offline Lock.

> **Llevalo a tu código:** Por cada patrón, encontralo en Hibernate: el contexto de persistencia es Identity Map más Unit of Work, y @Version es Optimistic Offline Lock. Anotá cada par en Obsidian.

Edición: Es de 2002, pero sigue siendo la referencia para entender cualquier ORM.



ESTACIÓN 13

☐ Leído

Domain-Driven Design

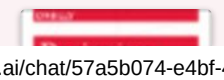
Eric Evans · ~560 págs

Cómo modelar un dominio complejo junto con los expertos del negocio: lenguaje ubicuo, entidades, agregados y contextos delimitados. Es denso; leé las partes I y II primero y después la IV (diseño estratégico).

> **Llevalo a tu código:** Armá el glosario de tu billetera con tu Manual de dominio FinTech al lado: Cuenta, Movimiento, Saldo, Transferencia. Definí cuál es el agregado y qué regla protege.

5 Sistemas distribuidos y producción

El corazón de la arquitectura FinTech: datos que no se pueden perder, servicios que fallan y dinero que no se puede duplicar.



ESTACIÓN 14

☐ Leído



Designing Data-Intensive Applications

Martin Kleppmann · ~600 págs

El libro central para FinTech: replicación, particionado, transacciones y niveles de aislamiento, consistencia y procesamiento de streams. Explica por qué un saldo se puede corromper.

> **Llevalo a tu código:** Reproducí un lost update en PostgreSQL con dos sesiones que descuentan del mismo saldo. Después arreglalo con SELECT ... FOR UPDATE y, aparte, con @Version.

Edición: En marzo de 2026 salió la 2ª edición, con Chris Riccomini como coautor. Si tu PDF es la 1ª (2017), sigue valiendo.



ESTACIÓN 15

☐ Leído

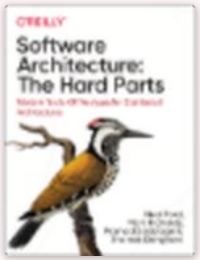
Building Microservices

Sam Newman · ~600 págs

Cuándo conviene partir un sistema, cómo hacerlo y qué cuesta: comunicación entre servicios, sagas, despliegue, testing y observabilidad.

> **Llevalo a tu código:** Partí Billetera Clara en dos servicios (cuentas y notificaciones) que se hablen por un evento, y anotá todo lo que se volvió más difícil.

Edición: La 2ª edición (2021) reescribe buena parte del libro. Es la que conviene.



ESTACIÓN 16

☐ Leído

Software Architecture: The Hard Parts

Ford, Richards, Sadalage y Dehghani · ~460 págs

Los dilemas de los sistemas distribuidos: cómo separar datos, transacciones distribuidas, sagas y contratos. Todo con un caso de estudio que sigue a un equipo durante el libro.

> **Llevalo a tu código:** Para una transferencia entre dos servicios, compará dos tipos de saga del libro y elegí una con su análisis de pros

y contras.



ESTACIÓN 17

Leído

Release It!

Michael Nygard · ~350 págs

Cómo falla el software en producción y cómo evitar que una falla tumbe todo: timeouts, circuit breaker, bulkheads.

> **Llevalo a tu código:** Simulá un proveedor de pagos lento y protegé la llamada con timeout y circuit breaker usando Resilience4j. Mirá qué pasa con y sin protección.

Edición: Conviene la 2ª edición (2018).



ESTACIÓN 18

Leído

System Design Interview

Alex Xu · vol. 1 · ~320 págs

Práctica de diseño de sistemas: rate limiter, generador de IDs únicos, sistema de notificaciones. Sirve para entrevistas de nivel medio y para aplicar lo de DDIA.

> **Llevalo a tu código:** Hacé cada capítulo primero vos en papel, 45 minutos, y recién después leé la solución del autor.



ESTACIÓN 19 · CONSULTA

Leído

Software Engineering at Google

Winters, Manshreck y Wright · ~600 págs

Cómo trabaja un equipo grande con código que dura años: code review, testing a escala, dependencias, documentación. Leé selectivamente, empezando por los capítulos de testing.

> **Llevalo a tu código:** Escribí una guía de estilo y de review de una

página para tu propio proyecto, como si entrara un compañero mañana.

Ramal IA

IA

Va por tu propio plan de IA, que arranca por matemáticas. Este libro es la parte de producto.



RAMAL IA

Leído

AI Engineering

Chip Huyen · 2025 · ~530 págs

Cómo construir aplicaciones sobre modelos fundacionales: evaluación, prompts, RAG, agentes, fine-tuning y optimización de inferencia. Es la parte de producto de tu plan de IA.

Llevalo a tu código: Armá un RAG mínimo sobre tus notas de Obsidian y medí si responde bien con 20 preguntas que escribas vos.

Ramal bajo nivel

BN

Para tus ratos libres: sistemas operativos, memoria y cómo corre realmente un programa.



RAMAL BAJO NIVEL

Leído

Programming from the Ground Up

Jonathan Bartlett · ~330 págs · gratis

Programar en assembly x86 sobre Linux: registros, memoria, la pila, llamadas al sistema y cómo se llama una función. Después de esto, la JVM deja de ser magia.

Llevalo a tu código: Hacé los ejemplos en tu Arch. Son de 32 bits, así que ensamblá con as

--32 y enlazá con ld -m elf_i386.

Edición: Es libre y se distribuye gratis.

TUS PDFS DE PRÁCTICA

Dónde encaja tu material propio

No son libros publicados, así que no los busqué en internet. Te marco en qué fase usar cada uno.



500 ejercicios de lógica

Calentamiento diario de 20 min.

FASE 1 → 3



Guía de reconstrucción Spring Boot + Maven

Tu método de reconstruir sin IA.

FASE 1



Grandes proyectos personales Java + Spring Boot

De acá sale el portfolio.

FASE 1 → 2



600 preguntas Java backend

Con tu carpeta de respuestas.

FASE 3



Prueba técnica Spring Boot desde cero

Hacela cronometrada, como una real.

FASE 3



Manual de dominio FinTech

Leelo junto con DDD.

FASE 4



1000 preguntas sistema bancario / startup

Repaso del dominio y del diseño.

FASE 4 → 5



Simulación proyecto final diplomatura FinTech

Usalo para aplicar DDIA y microservicios.

FASE 5

Fuera de la ruta: **claudecode-cheatsheet** es una herramienta, no lectura. **Quicksand_curso_completo** no lo pude identificar por el título; la miniatura parece un curso de diseño o tipografía, así que lo dejé afuera.

PARA SUMAR A LA CARPETA

Libros que te faltan

Elegidos por los huecos de tu biblioteca: tus puntos flojos (excepciones, streams, testing), la persistencia que usás todos los días y tus intereses de bajo nivel.

Java base · antes o durante la fase 1

Effective Java, 3ª ed.
Joshua Bloch
90 reglas cortas sobre Java bien escrito, incluidas excepciones, genéricos y lambdas. Es el libro que más te falta.

Modern Java in Action
Urma, Fusco y Mycroft
Streams, lambdas y Optional explicados con ejemplos. Ataca directo uno de tus puntos flojos.

Java Concurrency in Practice
Brian Goetz y otros
Hilos, locks y estado compartido. En FinTech dos transferencias al mismo tiempo son el caso normal.

Testing y persistencia · fase 2

Unit Testing Principles, Practices, and Patterns
Vladimir Khorikov
Qué testear, qué no y cuándo usar mocks. Ninguno de tus libros cubre testing de forma central.

High-Performance Java Persistence
Vlad Mihalcea
JPA, Hibernate y transacciones en serio: N+1, locking, niveles de aislamiento. Lo usás todos los días.

API Security in Action
Neil Madden
Autenticación, tokens y ataques a APIs, con ejemplos en Java. En finanzas la seguridad no es opcional.

Distribuidos y FinTech · fases 4 y 5

Microservices Patterns

Bajo nivel e IA · ramales

Computer Systems: A Programmer's

Chris Richardson

Sagas, eventos y consistencia con ejemplos en Java y Spring. Complementa a Newman con código.

System Design Interview, Vol. 2

Alex Xu y Sahn Lam

Trae diseños de un sistema de pagos y de una billetera digital, justo tu proyecto.

Kafka: The Definitive Guide

Shapira, Palino, Sivaram y Petty

Kafka está en tu ruta de repositorios; este libro te deja leer su código con contexto.

Perspective

Bryant y O'Hallaron

El siguiente paso natural después de Ground Up: memoria, caché, enlazado, procesos.

Operating Systems: Three Easy Pieces

Remzi y Andrea Arpaci-Dusseau

Sistemas operativos claros y gratis online. Lo podés practicar en tu Arch.

Crafting Interpreters

Robert Nystrom

Construís un lenguaje desde cero; el primer intérprete está escrito en Java. Gratis online.

Mathematics for Machine Learning

Deisenroth, Faisal y Ong

Encaja con tu decisión de empezar IA por matemáticas. Gratis en PDF.

Build a Large Language Model (From Scratch)

Sebastian Raschka

Para después de AI Engineering: construir un LLM pequeño entendiendo cada pieza.

PARA LLEGAR A SENIOR

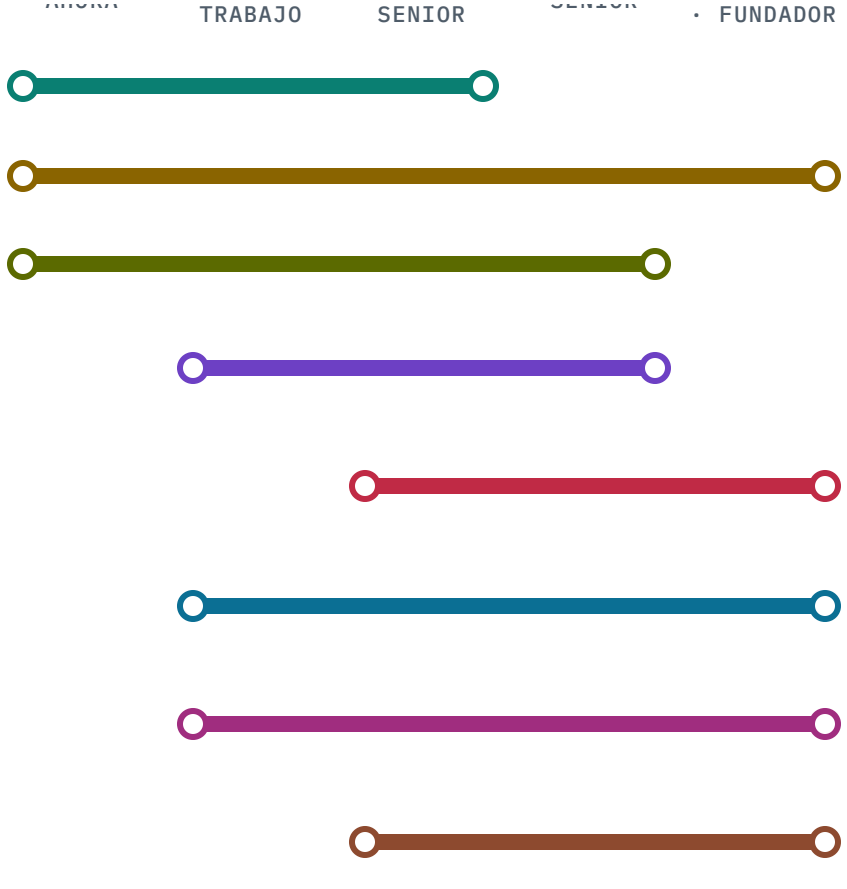
Lo que te falta para ser arquitecto senior FinTech

Tu ruta te deja como un backend que entiende arquitectura y sistemas distribuidos. Un arquitecto senior en finanzas además entiende cómo se mueve el dinero, lidera sin tener un cargo de jefe, escribe y negocia sus decisiones, y piensa en estrategia y producto. Estas ocho líneas suman 32 libros. No son todos para ahora: el mapa marca en qué momento de tu carrera entra cada línea, y en cada una hay un libro marcado para empezar.

LEÍDOS **0 / 32**

Cuándo entra cada línea

LÍNEA	AHORA	PRIMER	SEMI-	SENIOR	ARQUITECTO



Las etapas son aproximadas: si llegás antes a un puesto, adelantá la línea. Si podés, leelos en inglés; es práctica para trabajar con EE. UU. o la UE.

Oficio senior — AHORA → SEMI-SENIOR

The Pragmatic Programmer

Leído

David Thomas y Andrew Hunt · ed. 20º aniversario, 2019 · ~350 págs

Los hábitos de alguien que se hace cargo de lo que entrega: no repetir conocimiento, piezas que no dependen entre sí, prototipos que atraviesan todas las capas y automatizar lo que hacés a mano. Es de los pocos libros que casi todo senior leyó.

Probalo: Elegí algo que hacés a mano en Billetera Clara, como levantar la base o cargar datos de prueba, y dejalo en un script o en un perfil de Maven.

AHORA · CON LA FASE 2 EMPEZÁ POR ESTE

Dominio financiero — AHORA → ARQUITECTO

Accounting Made Simple

Leído

Mike Piper · ~120 págs

Contabilidad básica en un fin de semana: activo, pasivo, patrimonio y partida doble. Todo ledger financiero es partida doble, y con esto entendés por qué un saldo sale de sumar asientos en lugar de pisar un número.

Probalo: Escribí los asientos de una transferencia de $1.000 entre dos usuarios de tu billetera con una comisión de $10. Débitos y créditos tienen que sumar lo mismo.

Complemento gratis: la serie Accounting for

The Software Engineer's Guidebook

☐ Leído

Gergely Orosz · 2023 · ~400 págs

El mapa de la carrera de junior a staff, escrito por alguien que contrató y dirigió equipos en Uber: qué se espera en cada nivel y cómo es el trabajo diario de un senior.

Probalo: Leé el capítulo de tu nivel y el del siguiente. Anotá tres cosas del siguiente nivel que todavía no hacés y revisalas cada tres meses.

`PRIMER TRABAJO → SENIOR`

The Mythical Man-Month

☐ Leído

Frederick Brooks · ed. aniversario, 1995 · ~320 págs

Ensayos de 1975 que siguen describiendo proyectos de hoy. De acá salen la ley de Brooks (sumar gente a un proyecto atrasado lo atrasa más) y la integridad conceptual: un sistema necesita una visión coherente, y de esa idea nace el rol de arquitecto.

Probalo: Un equipo de n personas tiene n·(n−1)/2 canales de comunicación. Calculá cuántos se suman al pasar de 4 a 8 personas y pensá qué le pasa a cada reunión.

`FASE 4`

Developers de Modern Treasury.

`AHORA` `EMPEZÁ POR ESTE`

Analysis Patterns

☐ Leído

Martin Fowler · 1996 · ~360 págs · consulta

Modelos de dominio reutilizables. Sus capítulos de contabilidad (cuentas, asientos y reglas de asiento) son la base de cómo se modela un ledger con objetos. Es del mismo autor que Refactoring y PoEAA.

Probalo: Compará su modelo de cuentas y asientos con tu entidad Cuenta de apifintech. ¿El saldo es un campo que se pisa o la suma de los asientos? Probá la segunda versión.

`FASE 4 · CON DDD`

Payments Systems in the U.S.

☐ Leído

Benson, Loftesness y Jones · 3ª ed., 2017

Cómo se mueve el dinero de verdad: tarjetas, ACH y transferencias; autorización, compensación y liquidación; y quién cobra qué en cada paso. Es de EE. UU., pero esos conceptos te sirven para entender también Transferencias 3.0 en Argentina o Pix en Brasil.

Probalo: Dibujá el recorrido de un pago con tarjeta en un comercio: quién autoriza, quién compensa, quién liquida y cuándo le llega la plata al comercio.

`FASE 4 → 5`

The Ascent of Money

☐ Leído

Niall Ferguson · 2008 · ~450 págs

La historia del dinero, el crédito, los bonos, los seguros y las crisis. Lectura liviana para entender de dónde salen las instituciones con las que tu sistema se va a integrar.

Probalo: Por capítulo, anotá qué producto financiero de hoy nació de esa historia y qué entidad lo ofrece en Argentina.

CUANDO QUIERAS · LIVIANO

Fundamentals of Software Engineering for Fintechs

☐ Leído

Geison Goes · Leanpub · a evaluar

Su índice cubre justo tu objetivo: ledgers, idempotencia, sagas, compensación y liquidación, Pix, PCI, AML/KYC y conciliación. Es autopublicado, no encontré reseñas y no pude confirmar si está terminado, así que revisá el índice antes de pagarlo.

Probalo: Usá su índice como checklist del dominio: marcá cada tema que ya podrías explicar con un ejemplo de tu billetera.

FASE 5

Pensar y decidir

AHORA → SENIOR

Make It Stick

☐ Leído

Brown, Roediger y McDaniel · 2014 · ~310 págs

La ciencia de cómo se aprende y se recuerda. Explica por qué funciona tu método de reconstruir sin IA al día siguiente: es práctica de recuperación espaciada. Te sirve para cada libro de esta página.

Probalo: Antes de releer tus notas de Obsidian de un capítulo, escribí de memoria lo que te acordás. Recién después compará.

AHORA · EMPEZÁ POR ESTE

Thinking in Systems

☐ Leído

Donella Meadows · 2008 · ~240 págs

Arquitectura

PRIMER TRABAJO → SENIOR

Learning Domain-Driven Design

☐ Leído

Vlad Khononov · 2021 · ~340 págs

DDD explicado de forma moderna y mucho más fácil de leer que Evans. Suma cómo elegir el patrón de implementación según el tipo de subdominio, y event sourcing, que es como funcionan muchos ledgers.

Probalo: Clasificá las partes de tu billetera en subdominio núcleo, de soporte o genérico. Decidí cuál merece un modelo rico y en cuál alcanza con un CRUD.

FASE 4 · ANTES O JUNTO A EVANS

EMPEZÁ POR ESTE

Stocks, flujos y ciclos de realimentación: cómo se comporta un sistema entero más allá de sus partes. Sirve igual para un sistema distribuido, una economía o un equipo.

Probalo: Dibujá el ciclo de los reintentos: un servicio lento hace que los clientes reintenten, eso lo carga más y lo vuelve más lento. Marcá dónde lo cortarías.

SEMI-SENIOR

Thinking, Fast and Slow

☐ Leído

Daniel Kahneman · 2011 · ~500 págs

Los sesgos con los que decidimos todos: anclaje, exceso de confianza, falacia de planificación. Leelo sabiendo que algunos estudios del libro, como los de priming, no se pudieron replicar.

Probalo: Anotá tu estimación antes de cada tarea y el tiempo real al terminar. Después de diez tareas, calculá por cuánto te equivocás en promedio.

SEMI-SENIOR

Good Strategy Bad Strategy

☐ Leído

Richard Rumelt · 2011 · ~320 págs

Una estrategia real tiene tres partes: un diagnóstico, una política que guía y acciones coherentes entre sí. Te sirve para escribir la estrategia técnica de un equipo y la de tu futura startup.

Probalo: Escribí en una página la estrategia técnica de Billetera Clara para el próximo año con esas tres partes.

SENIOR

Building Evolutionary Architectures

☐ Leído

Ford, Parsons, Kua y Sadalage · 2ª ed., 2022 · ~260 págs

Cómo evitar que una arquitectura se degrade con los años usando funciones de aptitud: tests automáticos que verifican las reglas de arquitectura en cada build.

Probalo: Escribí un test con ArchUnit que falle si tu paquete de dominio importa algo de Spring o de JPA. Es la regla de Clean Architecture verificada en cada build.

FASE 4

Enterprise Integration Patterns

☐ Leído

Gregor Hohpe y Bobby Woolf · 2003 · ~680 págs · consulta

El vocabulario de la mensajería: canales, enrutadores, traductores y receptores idempotentes. En un banco casi todo es integrar sistemas viejos con nuevos, y Spring Integration y Apache Camel usan estos mismos nombres.

Probalo: Cuando conectes tus dos servicios por eventos en la fase 5, agregá un receptor idempotente: procesar dos veces el mismo evento no puede duplicar un movimiento.

FASE 5 · CONSULTA

Producción y seguridad

SEMI-SENIOR → ARQUITECTO

Site Reliability Engineering

☐ Leído

Beyer, Jones, Petoff y Murphy · Google · 2016 · gratis en sre.google

Cómo opera Google sus sistemas: objetivos de nivel de servicio (SLO), presupuestos de error, guardias y postmortems sin culpables. Un arquitecto FinTech diseña pensando en quién atiende la alarma a las 3 de la mañana.

Probalo: Definí un SLO de disponibilidad para las transferencias de Billetera Clara, por ejemplo 99,9 %, y calculá cuántos minutos por mes te deja fallar.

FASE 5 · EMPEZÁ POR ESTE

Observability Engineering

☐ Leído

Majors, Fong-Jones y Miranda, con Austin Parker · 2ª ed., junio 2026 · ~600 págs

Cómo entender qué pasa adentro de un sistema en producción a partir de eventos y trazas, más allá de las métricas. La 2ª edición es casi una reescritura y suma sistemas con IA.

Probalo: Instrumentá una transferencia con OpenTelemetry y seguí una sola operación de punta a punta: controlador, servicio, base de datos y evento.

FASE 5 · DESPUÉS DE SRE

Security Engineering

☐ Leído

Ross Anderson · 3ª ed., 2020 · ~1.200 págs · consulta

La referencia sobre cómo fallan los sistemas reales frente a atacantes. Tiene un capítulo entero de banca y contabilidad, con fraude, controles internos y separación de funciones.

Comunicación y negociación

PRIMER TRABAJO → ARQUITECTO

The Pyramid Principle

☐ Leído

Barbara Minto · escritura

Cómo escribir para gente ocupada: la conclusión primero y después los argumentos agrupados. Así se escriben las propuestas técnicas que se aprueban.

Probalo: Reescribí tu ADR de la fase 4 con la decisión en la primera oración. Si alguien lee solo esa línea, tiene que saber qué decidiste y por qué.

PRIMER TRABAJO · EMPEZÁ POR ESTE

Crucial Conversations

☐ Leído

Grenny, Patterson y otros · 3ª ed., 2021 · ~300 págs

Cómo sostener una conversación cuando hay mucho en juego, opiniones opuestas y emociones fuertes: un incidente en producción, un desacuerdo de arquitectura, un feedback difícil.

Probalo: Pensá en un desacuerdo técnico que tuviste y escribí qué querías realmente para vos, para el otro y para la relación, como propone el libro antes de hablar.

PRIMER TRABAJO

Communication Patterns

☐ Leído

Jacqui Read · 2023 · ~300 págs

Escrito para desarrolladores y arquitectos: diagramas pensados para quien los mira, documentación como código, ADRs y trabajo remoto entre husos horarios.

Probalo: Rehacé un diagrama de apifintech dos veces: uno para un desarrollador nuevo y

Completa la mirada de API Security in Action.

Probalo: Leé ese capítulo y revisá si tu billetera cumple separación de funciones: que una misma persona no pueda crear y aprobar el mismo movimiento.

FASE 5 · CONSULTA

otro para alguien de negocio. Anotá qué sacaste en cada versión.

FASE 4

Getting to Yes

☐ Leído

Fisher, Ury y Patton · 3ª ed., 2011 · ~200 págs

Negociar por intereses en vez de por posiciones, con criterios objetivos. Un arquitecto negocia plazos, alcance y deuda técnica todo el tiempo, y también te sirve para negociar tu sueldo.

Probalo: Antes de pedir tiempo para pagar deuda técnica, escribí qué le importa a la otra parte y qué vas a hacer si te dicen que no (lo que el libro llama BATNA).

SEMI-SENIOR

Liderazgo técnico

PRIMER TRABAJO → ARQUITECTO

Turn the Ship Around!

☐ Leído

L. David Marquet · 2012 · ~270 págs

Un capitán de submarino que dejó de dar órdenes y logró que su tripulación propusiera «tengo la intención de…». Es liderazgo sin depender del cargo, que es lo que hace un arquitecto.

Probalo: En el trabajo, en vez de preguntar qué hacer, avisá «tengo la intención de hacer X por esta razón» y mirá cómo cambia la conversación.

PRIMER TRABAJO EMPEZÁ POR ESTE

The Manager's Path

☐ Leído

Camille Fournier · 2017 · ~240 págs

Producto y startup

SEMI-SENIOR → ARQUITECTO

The Mom Test

☐ Leído

Rob Fitzpatrick · 2013 · ~130 págs

Cómo hablar con posibles clientes sin que te mientan por amabilidad: preguntá por lo que ya hicieron y evitá preguntar qué harían. Corto y práctico, para validar una idea de FinTech antes de programarla.

Probalo: Entrevistá a tres personas sobre cómo manejan hoy su plata, sin mencionar tu idea. Contá cuántas veces te dieron opiniones y cuántas te contaron hechos.

CUANDO TENGAS UNA IDEA EMPEZÁ POR ESTE

The Lean Startup

☐ Leído

Eric Ries · 2011 · ~320 págs

Camille Fournier · 2017 · ~240 págs

Aunque no quieras ser manager, los primeros capítulos explican cómo aprovechar a tu líder y cómo mentorear a otro, y el resto te muestra qué necesita un equipo de su tech lead.

Probalo: Llevá tus propios temas a la próxima reunión uno a uno con tu líder en lugar de esperar a que te pregunte.

`PRIMER TRABAJO`

Eric Ries · 2011 · ~330 págs

Construir, medir y aprender: lanzar un producto mínimo para probar una hipótesis antes de invertir meses. Hoy es vocabulario común en cualquier startup.

Probalo: Escribí la hipótesis más riesgosa de tu idea de startup y el experimento más barato que la pondría a prueba.

`ANTES DE FUNDAR`

Accelerate

☐ Leído

Forsgren, Humble y Kim · 2018 · ~260 págs

Investigación con datos de miles de equipos sobre qué prácticas hacen que una organización entregue software rápido y estable. Te da cuatro métricas para discutir con números en vez de opiniones.

Probalo: Medí las cuatro en tu repo: cada cuánto desplegás, cuánto tarda un cambio en llegar a producción, qué porcentaje falla y cuánto tardás en recuperarte.

`SEMI-SENIOR`

Inspired

☐ Leído

Marty Cagan · 2ª ed., 2017 · ~370 págs

Cómo trabajan los equipos de producto de las mejores empresas: descubrir qué construir antes de construirlo. Un arquitecto que entiende de producto diseña lo que el negocio necesita.

Probalo: Para una funcionalidad de tu billetera, evaluá los cuatro riesgos del libro: valor, usabilidad, factibilidad y viabilidad para el negocio. ¿Cuál es el mayor?

`SENIOR`

The Staff Engineer's Path

☐ Leído

Tanya Reilly · 2022 · ~330 págs

Qué hace un ingeniero de alto nivel que no es manager: ver el panorama grande, sacar adelante proyectos difíciles y subir el nivel de los demás. Es el camino técnico que termina en arquitecto.

Probalo: Para un proyecto de tu equipo, armá los tres mapas del libro: dónde está el equipo, cómo es el terreno y adónde quiere llegar.

`SEMI-SENIOR → SENIOR`

The Hard Thing About Hard Things

☐ Leído

Ben Horowitz · 2014 · ~300 págs

La parte de fundar que no aparece en los casos de éxito: despedir gente, quedarse sin plata, decidir con información incompleta. Para cuando tu startup deje de ser una idea.

Probalo: Anotá las tres decisiones más difíciles que imaginás para el primer año de tu startup y qué información te faltaría para tomar cada una.

`ANTES DE FUNDAR`

Team Topologies

☐ Leído

Matthew Skelton y Manuel Pais · 2ª ed., sept. 2025 · ~300 págs

La ley de Conway llevada a la práctica: la arquitectura termina copiando la forma de los equipos. Te da cuatro tipos de equipo y tres modos de interacción para diseñar las dos cosas juntas.

Probalo: Si tus servicios de la fase 5 tuvieran tres equipos, decidí cuál está alineado al flujo de negocio y cuál es de plataforma, y dibujá cómo se comunican.

SENIOR

The Software Architect Elevator

☐ Leído

Gregor Hohpe · 2020 · ~370 págs

El arquitecto como alguien que sube y baja entre la sala de máquinas y el directorio, traduciendo en los dos sentidos. Es un libro sobre el rol del arquitecto dentro de una organización grande.

Probalo: Explicá una decisión técnica de tu billetera dos veces: a un desarrollador, en términos de código, y a un gerente, en términos de riesgo y plata.

SENIOR → ARQUITECTO

Páginas aproximadas. El progreso se guarda solo en este navegador. Fuentes consultadas para ediciones vigentes:

- Kleppmann: DDIA 2ª edición
- O'Reilly: Fundamentals of Software Architecture, 2ª ed.
- Manning: Spring Start Here, 2ª ed.
- Manning: Spring in Action, 6ª ed.
- Beyond Cracking the Coding Interview
- O'Reilly: Head First Software Architecture
- Programming from the Ground Up
- AI Engineering, Chip Huyen
- IT Revolution: Team Topologies, 2ª ed.
- Honeycomb: Observability Engineering, 2ª ed.
- O'Reilly: Communication Patterns
- O'Reilly: Learning Domain-Driven Design
- Payments Systems in the U.S., 3ª ed.

- O'Reilly: Building Microservices, 2ª ed.
- O'Reilly: Release It!, 2ª ed.
- O'Reilly: Software Architecture: The Hard Parts
- Modern Treasury: Accounting for Developers
- Leanpub: Fundamentals of Software Engineering for Fintechs